

Overblik over estimeringsmetoder

To typer estimeringsmetoder og deres svagheder

Basalt set findes der to typer estimeringsmetoder: Erfaringsbaserede og modelbaserede. Ingen af de to typer er perfekte. Tværtimod har begge nogle ulemper.

De erfaringsbaserede metoder er kun gode, hvis fremtiden ligner fortiden. I og med at man anvender historiske data til estimering, kan man komme til at bygge sine estimater på data, der er forældede og irrelevante. Og hvis man bygger på udviklernes erfaring og erindring, så risikerer man, at vigtige ting er blevet glemt eller ligefrem fortrængt.

De model-baserede metoder, eller parameter-baserede som de også kaldes, bygger ofte også på historiske data. Typisk blev en række projekter analyseret ved hjælp af statistiske metoder for derved at finde de faktorer, som bidrager til omkostningerne (herunder udviklingsindsatsen) i et projekt. Signifikante faktorer blev fundet ved hjælp af korrelationsteknikker og blev så indarbejdet i en model ved hjælp af regressionsmetoder. Generelt blev modellernes koefficienter valgt, så man fik den bedst mulige overensstemmelse med en mængde af virkelige – men historiske – projekter.

Man kan sige, at de modelbaserede estimeringsmetoder især adskiller sig fra de erfaringsbaserede ved, at de explicit har forsøgt at bygge generelle modeller.

Overblik over erfaringsbaserede metoder

I afsnittene:

- Brug analogier og eksperter
- Brug historiske data

findes grundelementerne i erfaringsbaserede metoder beskrevet.

I afsnittet:

- Oppefra-ned versus nedefra-op estimering
- gennemgås et eksempel på, hvordan man ud fra et estimat af helheden kan bruge erfaringstal til at estimere de enkelte dele af projektet.

- Successiv kalkulation / Tre-punkts teknikken

er en avanceret estimeringsmetode, hvor erfaring især bruges til at skønne en mindsteværdi, en sandsynlig værdi, og en størsteværdi, for hver af de aktiviteter der estimeres. Teknikken kan med held kombineres med:

- Delfi-teknikken

en beslutningsstøtte-teknik, hvor man får en gruppe af eksperter (= masser af erfaring) til, ofte over 23 iterationer, at enes om et estimat.

Overblik over modelbaserede metoder

Klassikeren blandt modelbaserede metoder er:

- COCOMO

Udviklet i slutningen af 70'erne af Barry Boehm. I afsnittet beskrives både basal og intermediate COCOMO i detaljer og med eksempler.

COCOMO er netop i disse år ved at blive opdateret. Hvad det indebærer beskrives i afsnittet:

- COCOMO 2.0

En del af opdateringen indebærer, at man allerede i foranalysen kan give et estimat på udviklingsindsatsen ved at tælle skærm billeder og rapporter. Det findes beskrevet i afsnittene:

- Estimering af brugergrænsefladen
- Objekt point

En anden klassisk modelbaseret metode, der beskriver sammenhængen mellem indsatsen i projektet og tidspunktet i projektet, er beskrevet i afsnittet:

- Indsatsen afhænger af tidspunktet

Endelig findes i afsnittet:

- Function Points

beskrevet den modelbaserede metode, som de seneste år har fået mest opmærksomhed. I afsnittet beskrives både de oprindelige idéer, samt de nyeste udviklinger, herunder den internationale IFPUGstandard.

Brug analogier og eksperter

Analogier

Hvis en udvikler bliver bedt om et estimat for en eller anden aktivitet, vil vedkommende ofte helt intuitivt anvende en analogi. "Det modul ligner et, vi lavede i det sidste projekt, jeg var med til i den anden afdeling. Dengang tog det os to måneder for tre mand fuld tid, altså seks mandmåneder. Det her er nok lidt mindre, så mon ikke vi kan regne med fem mandmåneder", er et typisk udsagn, der illustrerer brugen af en analogi.

Etablering af en erfaringsdatabase til at forbedre brugen af analogier

Brugen af analogier er selvfølgelig forholdsvis begrænset for den enkelte, idet der naturligt er en grænse for, hvor mange projekter man kan have deltaget i. Derfor ville det være rart, hvis man kunne få adgang til andres erfaringer f.eks. i en database, så man i den kunne søge efter analoge projekteraktiviteter.

Etableringen og brugen af en sådan projektdatabase kan omfatte:

1. Etablering af en database, hvor både kvalitative erfaringer i form af tekst, samt kvantitative data, f.eks. om tid anvendt, fejl fundet el. lign. kan registreres. Desuden bør databasen understøtte søgninger både i fri tekst samt på nøgleord.
 2. Produktion af en procedure eller retningslinje for, hvordan data indsamles i projekter. Proceduren skal sandsynligvis tilpasses til udviklingsmiljøet. Der kan derfor godt blive tale om flere forskellige procedurer i samme virksomhed. F.eks. kan der være ret store forskelle mellem en afdeling, der implementerer standard SAP-økonomisystemer, og en afdeling der laver produktudvikling til et marked. Proceduren til SAP-økonomisystemer kan ret præcist indsamle data om hver af de standardmoduler der er i SAP, mens proceduren til produktudvikling må tage højde for, at megen produktudvikling er nyskabende og ikke kan sættes i bås.
 3. Projektledere og nogle udviklere skal trænes i brug af de procedurer, der er skrevet.
 4. Faktisk indsamling af data fra alle projekter. Her er det især vigtigt, at der ved afslutningen af ethvert projekt er afsat tid og ressourcer til at indsamle og rapportere erfaringer.
-

Eksperter

Eksperter er nogle, der ved hvad de snakker om. De besidder viden om et eller andet, f.eks. om hvordan en ny metode virker i praksis, eller om hvordan et konkret værktøj skal anvendes.

Det bør være eksperter, der primært leverer data til erfaringsdatabaser, for at sikre en høj kvalitet af data.

I mange projekter tager man nye værktøjer eller metoder i brug. Så snart et sådant projekt er afsluttet, vil man uvilkårligt blive organisationens "ekspert" i det nye. Det betyder, at næste gang et projekt står over for at skulle bruge værktøjet eller metoden, så vil det være naturligt at bede "eksperthen" om at deltage i estimeringen af projektet.

Tilsvarende vil man som projektleder ofte stå sig ved at se sig om i organisationen, eller blandt konsulenter, for derved at finde eksperter der kan hjælpe med estimeringen af ens projekt.

Man kan også systematisk samle en gruppe af eksperter og bruge:

- Delfi-teknikken

til at få estimeret sit projekt.

Brug historiske data

Historiske data kan være meget værdifulde

Historiske data kan bruges på meget avancerede måder til estimering. Man kan f.eks. opbygge en database, hvor man ikke alene registrerer estimeret og faktisk varighed af aktiviteter, men også karakteristika for aktiviteten (type, væsentligste problem, enkeltstående eller tæt koblet til andre, grænseflader etc.), samt hvem der var med i den gruppe, der gennemførte aktiviteten.

Basal brug af en database med historiske data går ud på at tage en aktivitet, søge i databasen, finde en lignende aktivitet, og derefter regne ud hvor lang tid (ud fra historiske tal) det vil tage at gennemføre aktiviteten. For eksempel:

- Databasen fortæller os, at en lignende aktivitet vil omfatte 300X linier kode
- Vi ved at vi kan lave et X linier kode på 1 mandtime
- Når et nyt program er 300X = 300 mandtimer, så tager det ca. 2 mandmåneder at gennemføre denne aktivitet

I afsnittet:

- Brug analogier og eksperter

finder du en gennemgang af, hvordan en organisation kan etablere en erfaringsdatabase, dvs. en database med historiske data.

Historiske data kan også bestå af målinger, og man kan kombinere disse målinger og derved få ret avancerede måleprogrammer ud af det.

Metrikker

En metrik er en kvantitativ karakteristik eller forskrift for måling af et eller flere aspekter ved et softwareprodukt eller ved softwareudviklingsprocessen.

Eksempler på simple målinger:

- Antal tusind linier kode (ofte forkortet KLOC)
- Antal fejlrapporter
- Antal kritiske fejl
- Antal timer brugt til test
- Antal siders dokumentation
- Brugertilfredshed på en skala fra 1 til 7
- Medarbejdertilfredshed på en skala fra 1 til 7
- Antal ændringer i kravspecifikation siden første baseline

Eksempler på sammensatte målinger

- Antal tusind linier kode / timer brugt på udvikling
- Faktisk leveringstid (i dage fra start) / estimeret leveringstid (i dage fra start)
- Defekter fundet ved reviews / defekter fundet ved test (som burde være fundet ved review)
- Antal ændringer i kravspecifikation set i forhold til brugertilfredshed

Man kan have stor glæde af ikke bare at indsamle kvalitative historiske data fra projekter, men også gennemføre målinger og indsamle kvantitative historiske data. Et eksempel på kvantitative historiske data, samt et eksempel på anvendelse af data, er vist nedenfor.

Historiske data for fremragende, effektive og normale projekter

Steve McConnell (1996) har indsamlet og bearbejdet historiske data fra en lang række kilder (Kemerer, 1987. Putnam & Myers 1992. Jones 1994). Ud af det er der kommet tre tabeller, som McConnell kalder fremragende, effektive og

nominelle erfaringsdata. Hver af tabellerne angiver nogle historiske data for systemsoftware (drivere, compilere), for in-house administrative systemer samt for standardprodukter f.eks. tekstbehandling.

Fremragende projekter - historiske data

Fremragende projekter ("Shortest Possible") er karakteriseret ved at:

1. Udviklerne på projektet (både analytikere og programmører) er blandt de 10% bedste i verden
2. Alle udviklere i projektet har mange års erfaring med programmeringssprog og programmeringsomgivelser
3. Alle har detaljeret viden om applikationsdomænet. Krav kendes fuldt ud ved projektstart, og de ændres ikke undervejs
4. Alle er højt motiverede, deler en fælles mission, og starter i projektet 100% fra projektets første dag, og er med hele vejen (så længe der er behov)
5. Der er gode værktøjer og moderne programmeringssprog til rådighed, der gennemføres aktiv risikostyring, der anvendes teknikker til hurtig udvikling ("Rapid Development"), og projektet gennemføres i et perfekt miljø med integreret brug af alle slags kommunikationværktøjer

KORTEST MULIG	Systemsoftware		Adm. In-house		Standard-produkt	
Systemstørrelse i linier kode	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder
10.000	6	25	3,5	5	4,2	8
15.000	7	40	4,1	8	4,9	13
20.000	8	57	4,6	11	5,6	19
25.000	9	74	5,1	15	6	24
30.000	9	110	5,5	22	7	37
50.000	11	230	7	46	8	79
70.000	13	350	8	71	9	120
100.000	15	540	9	110	11	190
200.000	20	1250	11	250	14	440
500.000	30	3900	17	780	20	1400

Effektive projekter - historiske data

EFFEKTIVE	Systemsoftware		Adm. In-house		Standard-produkt	
Systemstørrelse i linier kode	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder
10.000	8	24	4,9	5	5,9	8
15.000	10	38	5,8	8	7	12
20.000	11	54	7	11	8	18
25.000	12	70	7	14	9	23
30.000	13	97	8	20	9	32
50.000	16	190	10	40	11	67
70.000	19	290	11	61	13	100
100.000	22	450	13	93	15	160
200.000	29	1300	17	210	20	360
500.000	42	3100	25	640	29	1100

Effektive projekter er karakteriseret ved at:

1. Udviklerne på projektet (både analytikere og programmører) er blandt de 25% bedste i verden
2. Alle udviklere har erfaring med programmeringssprog og programmeringsomgivelser
3. Virksomheden – og dermed projektet - har en personaleudskiftning under 6% per år

4. Projektet har en fælles mission, og der er ingen åbne konflikter
5. Der er gode værktøjer og moderne programmeringssprog til rådighed, der gennemføres aktiv risikostyring, der anvendes teknikker til hurtig udvikling ("Rapid Development"), og projektet gennemføres i et perfekt miljø med integreret brug af alle slags kommunikationværktøjer

Typiske projekter - historiske data

Typiske ("Nominal") projekter er karakteriseret ved at:

1. Udviklerne på projektet (både analytikere og programmører) er blandt de 50% bedste i verden
2. Udviklerne på projektet har nogen erfaring med programmeringssprog og programmeringsomgivelser
3. Virksomheden – og dermed projektet – har en personaleudskiftning op til 10-12%
4. Der er nogle konflikter i og omkring projektet
5. Der er nogen brug af udviklingsværktøjer og moderne programmeringssprog
6. Der kun gennemføres begrænset risikostyring og brug af teknikker til hurtig udvikling ("Rapid Development")
7. Udviklingsmiljøet ikke er helt perfekt
8. Der kun er simple kommunikationsværktøjer (tlf., fax og elektronisk post) til rådighed

TYPISKE	Systemsoftware		Adm. In-house		Standard-produkt	
Systemstørrelse i linier kode	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder	Kalender måneder	Indsats i mandmåneder
10.000	10	48	6	9	7	15
15.000	12	76	7	15	8	24
20.000	14	110	8	21	9	34
25.000	15	140	9	27	10	44
30.000	16	185	9	37	11	59
50.000	20	360	11	71	14	115
70.000	23	540	13	105	16	175
100.000	26	820	15	160	18	270
200.000	35	1900	20	370	24	610
500.000	51	5500	29	1100	35	1800

Oppefra-ned versus nedefra-op estimering

Oppefra-ned estimering

Ved oppefra-ned estimering, eller top-down estimering som det ofte kaldes, starter man med et estimat for hele projektet. Dernæst bruger man erfaringsdata omhandlende hvor lang tid hver del af et projekt tager, til at dele det overordnede estimat op i en række estimater for hver aktivitet i projektet.

Nedenfor er vist erfaringsdata fra USA for hvor stor en procentdelen af indsatsen i et projekt, der anvendes i hver af 23 faser.

Erfaringsdata fra USA indsamlet af Capers Jones

Capers Jones har indsamlet data fra mere end 6700 amerikanske projekter (1996: side 141). Nedenfor er vist et uddrag af tabel 3.14 (side 184) omhandlende "Software Development Activities Associated with Six Sub-industries (Percentage of Staff-months by Activity)".

Aktivitetstype	MIS	Outsourced	Systems
1. Krav	7,5	9	4
2. Prototyping	2	2,5	2
3. Arkitektur	0,5	1	1,5
4. Projektplanlægning	1	1,5	2
5. Initialt design	8	7	7
6. Detaljeret design	7	8	6
7. Design review	-	0,5	2,5
8. Kodning	20	16	20
9. Genbrugs-klargøring	-	2	2
10. Indkøb af tredje-parts software	1	1	1
11. Kodeinspektion	-	-	1,5
12. Konfigurationsstyring	3	3	1
13. Formel integration	2	2	2
14. Brugerdokumentation	7	9	10
15. Modultest	4	3,5	5
16. Funktionstest	6	5	5
17. Integrationstest	5	5	5
18. Systemtest	7	5	5
19. Test i "marken" (Field test)	-	-	1,5
20. Accepttest	5	3	1
21. Kvalitetssikring	-	1	2
22. Installation & træning	2	3	1
23. Projektledelse	12	12	12

Der er vist et uddrag for tre industrier, svarende til de tal der er mest relevante for medlemmerne af Datateknisk Forum:

1. "MIS" lig med Management Information Systems. Gruppen omfatter alle former for administrative edbsystemer, store mainframe såvel som små PC-baserede, udviklet til internt brug.
2. "Outsourced" omfatter software produceret på kontrakt, hvad enten der er tale om fast pris, timer & materialer, eller andre kunde-leverandør forhold.
3. "Systems" omfatter dels systemsoftware f.eks. operativsystemer, oversættere, og lokånet, dels indlejret software til apparater, telefonsystemer, luftfart el. lign. Hovedparten af software i dansk elektronikindustri falder i denne gruppe.

Hvor valide er Capers Jones data?

Som nævnt har Capers Jones indsamlet data fra mere end 6700 projekter. To femtedele af data er indsamlet af medarbejdere eller klienter fra den organisation Capers Jones kommer fra. Lidt over en tredjedel af data stammer fra baglænsregning ("backfiring") efter afslutningen af et projekt, f.eks. ud fra antal linier kode. Endelig kommer ca. en femtedel af data fra andre organisationer, der selv har rapporteret f.eks. til International Function Point User Group (IFPUG).

Det er klart, at data fra så mange kilder kan være behæftet med ganske store fejl. Det ved Capers Jones udmærket, men som han siger (1996: side 143): *"Errors can be corrected by subsequent researchers, but if the data is not published there is no incentive to improve it."*

For de tre rækker tal ovenfor angiver Capers Jones (1996, side 147-148) følgende primære fejlkilder:

1. "MIS" er den mest fejlbehæftede gruppe af tal. Fejlene stammer fra ubetalt overtid, manglende registrering af ledelsesomkostninger, manglende registrering af brugernes tid, samt manglende registrering af brugen af specialister f.eks. til kvalitetssikring.
2. "Outsourced" er en gruppe med meget pålidelige omkostningstal, selv om overtid ofte er ubetalt og dermed udeladt.
3. "Systems" er også en gruppe meget pålidelige tal, selv om overtid også her tit er udeladt. Specielt er data for test og kvalitetssikring ofte gode for denne gruppe.

Alt i alt skal man selvfølgelig tage Capers Jones tal med et vist forbehold. På den anden side er det nok den mest detaljerede registrering, der findes. Så for nuværende fås tallene ikke bedre, med mindre man selv indsamler erfaringstal i egen organisation.

Nedefra-op estimering

Det program, der skal laves, opdeles i moduler. For hvert programmodul estimeres, hvilken indsats modulet kræver.

Ved estimeringen af modulerne inddrages så vidt mulig den gruppe af udviklere, der faktisk skal udvikle modulerne. Derved udnyttes at programmører som regel er ret gode til at estimere netop programmeringsindsats, mens erfaringen viser, at programmører sjældent er gode til præcist at estimere andre aktiviteter.

Dernæst tages der stilling til, hvilke andre aktiviteter projektet omfatter. Hver af disse aktiviteter estimeres.

Til sidst kan man sammenligne listen af aktiviteter og estimerer med de procenter, som er gengivet fra Capers Jones ovenfor. Hvis dine egne estimerer matcher med det typiske, så har du en god indikator for, at dine estimerer ikke er helt ved siden af. Er der derimod enkelte aktiviteter, der er estimeret til at være meget større eller mindre end det typiske, så bør der være en god projektspecifik forklaring, eller måske bør ens estimat gø overvejes.

Test og projektledelse er ofte glemte aktiviteter

De aktiviteter, som mindre rutinerede projektledere oftest glemmer, er tid nok til test (ca. 25%) og tid nok til projektledelse og -planlægning (13-14%). Derfor er det en god idé at gøre en del ud af at sikre, at estimererne af indsats til disse aktiviteter er særligt velunderbyggede.

Successiv kalkulation / tre-punkts teknikken

Tankegangen bag successiv kalkulation

“Enhver forkalkulation beskæftiger sig ... med fremtidige forhold og indeholder derfor nødvendigvis en række skøn, der alle er behæftet med en vis usikkerhed”

“Et gammel grundprincip ved kalkulationsarbejde er, at man tilstræber at opdele kalkulationen i et mindre antal hovedposter og herefter opdele de mest betydningsfulde af disse i delposter, som så igen - afhængig af den ønskede nøjagtighed - opdeles yderligere.”

“Ved ... at anvende nogle statistiske grundbegreber er det lykkedes at opstille en metode, der synes at kunne gøre meget kalkulationsarbejde såvel mere effektivt som hurtigere.”

Kilde: Steen Lichtenberg (1971). Rapport over successiv kalkulation. DtH. Citeret fra forordet.

For hver aktivitet gives tre estimater

For hver aktivitet skal der angives tre estimater:

V_o = mest optimistiske skøn (mindsteværdi)

Det estimat, hvor man ikke kan bevise, at man ikke vil være færdig med opgaven. Der tages udgangspunkt i de bedst mulige betingelser og ideelle omgivelser som udgangspunkt for estimatet.

V_s = mest sandsynlige værdi

Det bedste gæt på, hvor lang tid opgaven vil tage, når vi gør, som vi plejer og støder på de sædvanlige forhindringer og forsinkelser.

V_p = mest pessimistiske værdi (størsteværdi)

Det estimat, hvor der er taget højde for næsten alle tænkelige forhindringer og forsinkelser. ”Murphy arbejder med hele vejen” (Jf. Murphys love). Der er 99% sandsynlighed for, at opgaven er løst inden for den estimerede størsteværdi.

De tre estimater bruges til at opstille en fordelingsfunktion

Når man tager udgangspunkt i, at menneskets evne til at forudsige egen ydeevne er en sandsynlighedsfunktion, så kan man bruge de tre estimater – mindsteværdi, sandsynlig værdi, størsteværdi – til at opstille en fordelingsfunktion.

Sandsynlighedsfunktionen kan antages at være en Erlang-funktion

Den danske matematiker A.K.Erlang opstillede under sit arbejde med telefonsystemer et antal fordelingsfunktioner. For en Erlang funktion med en k -værdi på 10, hvor “ k ” er et udtryk for fordelingsfunktionens skævhed, kan middelværdien V_m beregnes som:

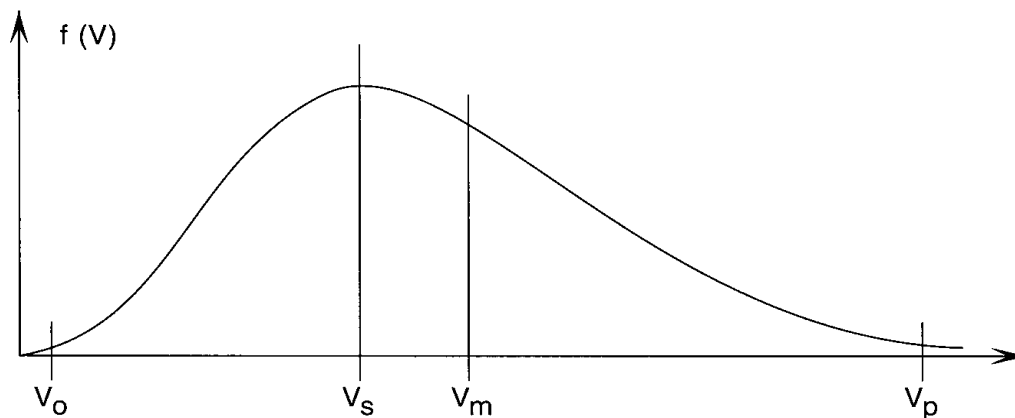
$$V_m = (V_o + 2,95 \cdot V_s + V_p) / 4,95$$

Og standardafvigelsen “ S ” kan beregnes som:

$$S^2 = (V_p - V_o)^2 / 21,66$$

Steen Lichtenberg (1971: side 20) argumenterer for valget af $k = 10$ på følgende måde: "... fordi den må anses for at være mest repræsentativ. For andre k -værdier er ovennævnte udtryk for middelværdien behæftet med en fejl, der dog for de mere hyppigt forekommende k -værdier ($k = 5$ til 15) kun når op til få promille."

FORDELINGSKURVE



Tilnærmede formler til beregning af middelværdi og standardafvigelse

Som tilnærmede formler kan vi bruge:

$$\text{Middelværdien } V_m = (V_0 + 3 \cdot V_s + V_p) / 5$$

$$\text{Standardafvigelsen } S = (V_p - V_0) / 5$$

Sandsynligheden for at være færdig inden for en given tid

Middelværdien og standardafvigelsen kan bruges til at beregne et estimat med en vis sandsynlighed for færdiggørelse. For Erlang-fordelingsfunktionen med $k=10$ gælder, at 68% af alle udfald er omfattet i intervallet fra middelværdien plus-minus standardafvigelsen:

$$68\% \text{ er omfattet. Formel: } (V_m - S) - (V_m + S)$$

Eller sagt på en anden måde så er middelværdien plus standardafvigelsen den værdi, hvor man med 85% sandsynlighed er færdig med aktiviteten.

For Erlang-fordelingsfunktionen med $k=10$ gælder, at 95% af alle udfald er omfattet i intervallet fra middelværdien plus-minus to gange standardafvigelsen:

$$95\% \text{ er omfattet. Formel: } (V_m - 2S) - (V_m + 2S)$$

Eller sagt på en anden måde så er middelværdien plus to gange standardafvigelsen den værdi, hvor man med 97% sandsynlighed er færdig med aktiviteten.

For Erlang-fordelingsfunktionen med $k=10$ gælder at 99% af alle udfald er omfattet i intervallet fra middelværdien plus-minus tre gange standardafvigelsen:

$$99\% \text{ er omfattet. Formel: } (V_m - 3S) - (V_m + 3S)$$

Eller sagt på en anden måde så er middelværdien plus tre gange standardafvigelsen den værdi, hvor man med 99,5% sandsynlighed er færdig med aktiviteten.

Eksempel på beregning af et timetal med 85% sandsynlighed

For et nyt skærm billede i et program har du skønnet:

V_o (mindsteværdi) = 70 timer

V_s (mest sandsynlig) = 80 timer

V_p (størsteværdi) = 110 timer

Middelværdien V_m kan nu beregnes som:

$$V_m = (V_o + 3 \cdot V_s + V_p) / 5 = (70 + 3 \cdot 80 + 110) / 5 = 84$$

Standardafvigelsen S kan beregnes som:

$$S = (V_p - V_o) / 5 = (110 - 70) / 5 = 8$$

For Erlang-fordelingsfunktionen gælder, at 68% af alle udfald er omfattet i intervallet fra middelværdien plusminus standardafvigelsen:

$$68\% \text{ er omfattet: } [(V_m - S) \rightarrow (V_m + S)] = [(84-8) \rightarrow (84+8)] = [76 \rightarrow 92]$$

Og de 92 timer, lig med middelværdien plus standardafvigelsen, er den værdi, hvor man med 85% sandsynlighed er færdig med aktiviteten.

Hvilket estimat skal man melde offentligt ud ?

Tom DeMarco foreslår i bogen "Controlling Software Projects" (1982), at man bruger middelværdien som definitionen af et estimat.

I flere danske virksomheder, bl.a. B&O og Brüel & Kjær, har man som estimat brugt middelværdien plus standardafvigelsen ($V_m + S$), dvs. den tid man har 85% sandsynlighed for at være færdig inden.

Andre steder har man lagt stor vægt på at angive estimer som intervaller. F.eks. kunne man have meldt ud, at det vil tage mellem 2000 og 2800 timer at gennemføre en given aktivitet. Og samtidig har man f.eks. fortalt styregruppen, at intervallet var 95% konfidens-intervallet ($[V_m - 2S \rightarrow V_m + 2S]$).

I fastpris-projekter, hvor leverandøren jo bærer en meget stor del af risikoen, har jeg mødt konsulentvirksomheder og softwarehuse, der bruger den tid man har 97% sandsynlighed for at være færdig inden ($V_m + 2S$), som det estimat der ligger til grund for et fastpristilbud.

Standardafvigelse for et projekt med flere aktiviteter

Ud fra de tre værdier – mindsteværdi, sandsynlig værdi, størsteværdi – kan man beregne middelværdi og standardafvigelse for en aktivitet. For at kunne finde den samlede standardafvigelse for et projekt med flere aktiviteter er man nødt til at bruge variansen.

Varians " V " er lig med kvadratet af standardafvigelsen:

$$V = S^2$$

Standardafvigelsen S_{total} for et projekt med flere aktiviteter er lig med kvadratroden af summen af varianser for hver enkelt af projektets aktiviteter:

$$S_{\text{total}} = \sqrt{\sum V_{\text{aktivitet 1 til } n}}$$

Eksempel på beregning for et projekt med flere aktiviteter

Vi antager, at et projekt består af fem delaktiviteter kaldet Alfa, Bravo, Charlie, Delta og Ekko.

	V_o	V_s	V_p	V_m	S	V
Alfa	20	25	35	26	3	9
Bravo	60	85	115	86	11	121
Charlie	55	65	100	70	9	81
Delta	15	20	30	21	3	9
Ekko	35	50	65	50	6	36
I alt				253	$\Sigma V =$	256
				$\sqrt{\Sigma V} =$	16	

For hver af de fem delaktiviteter har vi skønnet en mindsteværdi, den mest sandsynlige værdi, og en størsteværdi. Derefter har vi beregnet middelværdi, standardafvigelse og varians.

Aktiviteten med den største usikkerhed nedbrydes yderligere

Idéen i successiv kalkulation er, at den aktivitet, der har den største usikkerhed nedbrydes yderligere. I eksemplet ovenfor er det aktiviteten Bravo, der har den største usikkerhed (standardafvigelse). Vi nedbryder derfor denne aktivitet yderligere, f.eks. i Bravo-1, Bravo-2 og Bravo-3 som vist nedenfor.

	V_o	V_s	V_p	V_m	S	V
Alfa	20	25	35	26	3	9
Bravo-1 20	25	40	27	4	16	
Bravo-2 30	35	45	36	3	9	
Bravo-3 15	25	30	24	3	9	
Charlie	55	65	100	70	9	81
Delta	15	20	30	21	3	9
Ekko	35	50	65	50	6	36
I alt				254	$\Sigma V =$	169
				$\sqrt{\Sigma V} =$	13	

Nu er det aktiviteten Charlie, der tegner sig for den største usikkerhed. Derfor nedbryder vi aktiviteten Charlie i de tre aktiviteter Charlie-1, Charlie-2, og Charlie-3.

	V_o	V_s	V_p	V_m	S	V
Alfa	20	25	35	26	3	9
Bravo-1 20	25	40	27	4	16	
Bravo-2 30	35	45	36	3	9	
Bravo-3 15	25	30	24	3	9	
Charlie-1	25	35	60	38	7	49
Charlie -2	20	20	30	22	2	4
Charlie -3	6,33	11,22	15	11	1,73	3
Delta	15	20	30	21	3	9
Ekko	35	50	65	50	6	36
I alt				255	$\Sigma V =$	144
				$\sqrt{\Sigma V} =$	12	

Fortsæt nedbrydningen indtil ...

Steen Lichtenbergs tommelfingerregel for, hvor længe nedbrydningen skal fortsætte, er (1971: side 14): “Den eller de poster der har den største varians ... opdeles i et antal dδposter ... Således fortsættes indtil man enten har nået en tilfredsstillende nøjagtighed eller møder usikkerheder, der ikke kan nedbringes ved opdeling eller på anden vis.”

En anden tommelfingerregel, der bygger på konkret erfaring fra udvikling af administrative edb-systemer er: (Willkan, 1998): “Bliv ved med at nedbryde aktiviteterne indtil standardafvigelsen er mindre end en tyvendedel af middelværdien: $S < V_m / 20$ ”

I eksemplet ovenfor ville det betyde, at vi kunne stoppe nedbrydningen efter tredje omgang, idet 12 er mindre end $(255 / 20)$.

Ofte vil det i praksis være urealistisk at nå ned på så små usikkerheder som en tyvendedel, især tidligt i et projekt.

I afsnittet om:

- Risiko og usikkerhed

kan du læse mere om usikkerhed på forskellige tidspunkter i projektet.

Successiv kalkulation forudsætter mindst 30 uafhængige opgaver

For at benytte Erlang-fordelingsfunktionen kræves en opdeling af projektet i mindst 30 aktiviteter. Ellers er det teoretiske grundlag for, at fordelingsfunktionen med god tilnærmelse kan bruges ikke til stede. Successiv kalkulation giver kun korrekte resultater, såfremt de enkelte aktiviteter er statistik uafhængige af hinanden. Et forhold, der påvirker usikkerheden på flere aktiviteter, må derfor håndteres ved at lade usikkerhedsforholdet indgå som en særskilt virtuel aktivitet.

Eksempel på håndtering af fælles usikkerhed som en virtuel aktivitet

	V _o	V _s	V _p	V _m	S	V
Alfa	20	25	35	26	3	9
Bravo	60	85	115	86	11	121
Charlie	55	65	100	70	9	81
Delta	15	20	30	21	3	9
Ekko	35	50	65	50	6	36
Usikkerhed vedr. CASE-værktøj	-50	0	100	10	10	100
I alt				263		ΣV= 356
					√ ΣV= 18,87	

Her er vist et eksempel på, hvordan ibrugtagningen af et nyt CASE-værktøj, et forhold der selvfølgelig ofte vil påvirke mange projektaktiviteter, er trukket ud som en selvstændig aktivitet.

I eksemplet antages, at CASE-værktøjet i bedste fald vil spare os for 50, og i værste fald vil tage 100 ekstra, f.eks. pga. indlærings tid, begynderfejl etc. Vi skønner dog, ud fra tidligere erfaringer med værktøjer i den pågældende organisation, at fordelene mest sandsynligt vil gå lige op med ulemperne.

Delfi-teknikken

Delfi-teknikken er en gruppe-beslutningsteknik

Delfi-teknikken er en teknik for grupper. Kort beskrevet anvendes den på følgende måde:

1. Man samler en gruppe.
 2. Hvert medlem af gruppen estimerer den samme aktivitet.
 3. Derefter sammenligner man resultaterne.
 4. Ligger resultaterne tæt på hinanden kan man umiddelbart bruge middelværdien som estimat.
 5. Hvis estimerterne derimod ligger langt fra hinanden, så kan man enten bruge den accelererede eller den traditionelle Delfi-teknik.
-

Den traditionelle Delfi-teknik

Den traditionelle Delfi-teknik kaldes i litteraturen ofte for "Wideband Delphi". Teknikken går ud på følgende:

1. Find først et antal personer med ekspertise eller erfaring inden for de aktiviteter som estimeringen drejer sig om.
2. Opnå personernes accept af at deltage i en gruppe-estimerings-session.
3. Find et tidspunkt og et mødested, der passer alle gruppemedlemmer.
4. Udlever en liste over aktiviteter, der skal estimeres. Udleverer evt. samtidig skema eller skabelon, der skal bruges til estimeringen.
5. Projektets baggrund, formål, indhold, vision og begrænsninger præsenteres for gruppemedlemmerne.
6. Den aktivitet, der skal estimeres, gennemgås kort.
7. Hvert gruppemedlem gennemfører estimering af aktiviteten uden at diskutere med andre. Begrundelser for estimatet noteres omhyggeligt.
8. Mødelederen indsamler estimerterne. Beregner en middelværdi og viser estimerterne i anonymiseret form i et diagram.
9. Gruppen gennemgår diagrammet og forsøger at forklare forskellene.
10. Hvert enkelt gruppemedlem opdaterer sit eget estimat ud fra forudgående diskussion.
11. Mødelederen indsamler igen estimerterne. Hvis estimerterne konvergerer stoppes her. Ellers diskuteres og estimeres endnu en gang.

Traditionelt gør man en del ud af at sikre de enkelte gruppemedlemmers anonymitet, når man bruger Delfiteknikken. Derved sikres, at der ikke er et gruppemedlem, der dominerer de andre, hvorved fordelene ved at være en gruppe forsvinder.

Et gruppemedlem kan opnå dominans ved at have meget længere erfaring, højere placering i organisationshierarkiet, eller meget aggressiv adfærd. Er en eller flere af disse forhold til stede, kan man bruge en facilitator og/eller elektronisk post og/eller et særligt indrettet gruppe-beslutnings-støtte laboratorium til at sikre anonymiseret estimering. Hvis der ikke er nogen grund til at tro, at et enkelt gruppemedlem vil være dominerende, så kan det bedre svare sig at anvende den accelererede Delfi-teknik.

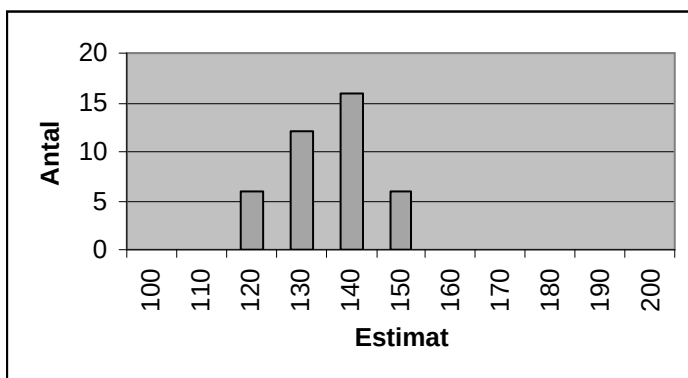
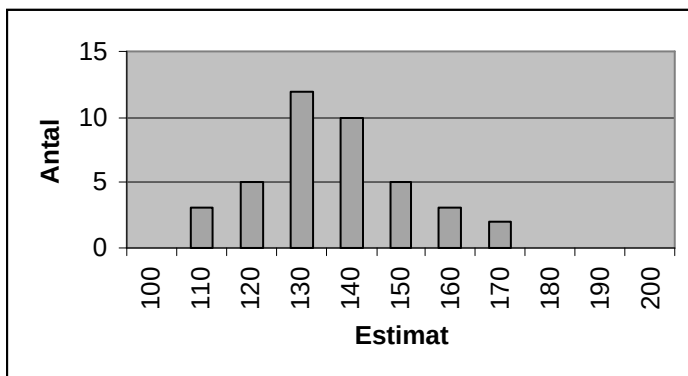
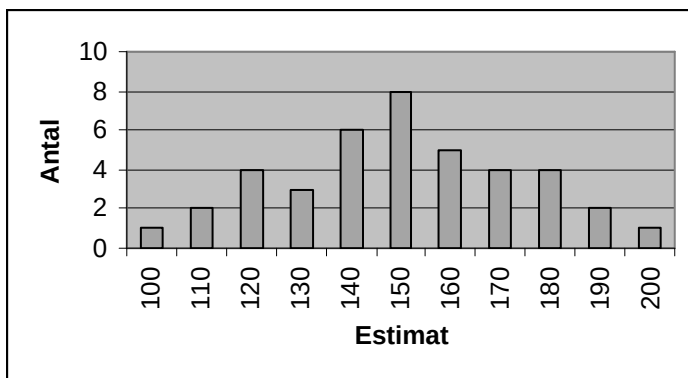
Den accelererede Delfi-teknik

Punkt 1. til 7. gennemføres lige som i den traditionelle Delfi-teknik. Derefter sker der følgende:

8. Hvert medlem af gruppen fortæller hvorfor og hvordan, de er nået frem til deres estimat. Derved vil de medlemmer af gruppen, der lå lavt, blive klar over forhold, de ikke har taget højde for, og de medlemmer af gruppen der lå højt, vil indse, at de måske var for pessimistiske.
9. Efter at alle har fortalt om baggrunden for estimerterne, estimerer hvert gruppemedlem på ny den samme aktivitet. Alt andet lige vil estimerterne nu ligge meget tættere på hinanden, således at middelværdien kan bruges som gruppens estimat.

10. I særlige tilfælde kan det være nødvendigt med tre eller fire omgange, hvor man fortæller hinanden om baggrunden for henholdsvis lave og høje estimater.

Nedenfor er vist tre histogrammer, der kunne være en illustration af udviklingen i en gruppes estimater efter første, anden og tredje estimeringsrunde.



Delfi-teknikken fungerer fint med grupper på 3-5 personer. Ved store eller særligt komplicerede projekter kan man anvende meget større grupper. Her kan man nøjes med at lade de gruppemedlemmer, der ligger i nederste og øverste kvartil fortælle resten af gruppen, hvorfor deres estimat blev som det blev.

Estimering af antal linier kode

Linier kode: Udbredt og udskældt

Antallet af kodelinier er den ældste og mest udbredte måde at angive størrelsen af et program på. Den anvendes inden for langt de fleste områder, bortset måske fra administrative IT-systemer, hvor Function Points efterhånden har vundet terræn.

Antallet af kodelinier er også noget af det mest udskældte, man kan finde inden for IT-verdenen. Nogle af argumenterne mod at tælle kodelinier er:

1. Det er upræcist, hvad en linie er - skal kommentarer f.eks. tælles med ?
2. Belønner man ikke ineffektivitet: Hvis en programmør kan løse et problem på 100 linier på tre dage, mens en anden skal bruge 300 linier og fem dage, så vil sidstnævnte programmør fremstå som mest produktiv!
3. Hvis man bruger automatisk kodegenerering, visuelle programmeringssprog, gøbrugsmoduler af ukendt størrelse m.v. så er antallet af kodelinier både ukendt og irrelevant.
4. Man kan ikke sammenligne på tværs af programmeringssprog, idet en kodelinie i C++ kan gøre det samme som to eller tre linier af et andet programmeringssprog. Og hvis man alligevel forsøger, så får man den kontraintuitive effekt, at jo højere niveau programmeringssprog man anvender, jo mindre effektive er programmørerne målt i antal kodelinier per tidsenhed.

Standard for at tælle kodelinier

Efterhånden er man enedes om regler for, hvordan kodelinier tælles. Kort sagt så tæller man:

1. Logiske kodelinier. F.eks. tælles en linie C++ med to sætninger adskilt af semikolon som to kodelinier, selv om det fysisk kun er en.
2. Udførbare sætninger og data-definitioner.
3. Bruger-definerede "include files" én gang.
4. Jobafviklings-linier ("Job Control language" - JCL).
5. SQL sætninger.
6. *Ikke* kommentarer.
7. *Ikke* blanke linier.
8. *Ikke* kode der gentages mange gange (f.eks. "include files"). Det tælles kun med første gang.
9. *Ikke* standard styresystem "include files".
10. *Ikke* kode til at understøtte support, test eller vedligeholdelse. Disse linier kan evt. tælles separat.

Der findes ikke nogen officiel standard, f.eks. fra ISO, for kodelinietælling. Capers Jones (1986: 403411) har skrevet en detaljeret "Rules for Counting Procedural Source Code", som kan anbefales som udgangspunkt for en virksomhedsspecifik procedure - hvis man skulle have brug for en sådan.

Fra kodelinier til estimat

Undersøgelser har vist, at programmører faktisk er ret gode (præcise) til at estimere antallet af kodelinier i et modul, de skal skrive. Derfor gælder det om at komme fra en overordnet arbejdsstruktur til en konkret opdeling af produktet i moduler eller objekter, evt. via nogle arkitektur-overvejelser.

Når produktet, der skal laves, er opdelt i moduler, så kan man let bruge analogier til at bestemme størrelsen af nogle moduler. "Det modul ligner et vi lavede i det sidste projekt, jeg var med til. Det fyldte dengang 1200 linier. Det gør dette nok også" er et udsagn, der illustrerer anvendelsen af en analogi.

Når produktet, der skal laves, er opdelt i moduler, så har man også den fordel, at man kan bedømme den person eller den lille gruppe, der faktisk skal stå for programmeringen, om at estimere størrelsen.

Der er ikke noget i vejen for, at man anvender

- Successiv kalkulation *også kaldet* tre-punkts estimering

til at estimere antal kodelinier. Så skal man blot bede om at få tre værdier for hvert modul:

1. V_o = mest optimistiske skøn (mindsteværdi). Det estimat, hvor man ikke kan bevise, at det kan gøres på færre kodelinier.
2. V_s = mest sandsynlige værdi. Det bedste gæt på, hvor mange linier modulet vil omfatte, når vi gør som vi plejer.
3. V_p = mest pessimistiske værdi (størsteværdi). Det estimat af antal kodelinier, hvor der er 99% sandsynlighed for at opgaven er løst inden for den estimerede størsteværdi.

Endelig kan man samle en gruppe, evt. omfattende nogle eksperter og så anvende:

- Delfi-teknikken

til at estimere antallet af kodelinier.

Delfi-teknikken kan også anvendes til at estimere antallet af kodelinier på et tidligere tidspunkt end når modulopdelingen af produktet er kendt. F.eks. kunne man bede en gruppe give det første grove estimat ud fra projektets hovedaktiviteter som et antal linier kode.

Når antallet af estimerede kodelinier kendes for hvert modul, så kan antallet af linier summeres og bruges som input til metoderne:

- COCOMO, eller
 - COCOMO 2.0
-

COCOMO

Constructive Cost Model (COCOMO)

COCOMO-modellen blev først beskrevet af Barry Boehm i bogen Software Engineering Economics fra 1981. Den første COCOMO havde tre niveauer: Basal, Detaljeret og "Intermediate" (midt i mellem). Basal COCOMO vandt en del udbredelse, og den citeres ofte. Intermediate COCOMO, der tager højde for 15 situationer og projekt-specifikke faktorer, vandt også en del udbredelse, mens den detaljerede COCOMO model, der tager højde for fasespecifikke faktorer aldrig har vundet større udbredelse (jf. Legg 1997). Nedenfor gennemgås derfor kun basal og intermediate COCOMO.

Fordele ved COCOMO frem for andre estimeringsteknikker

Fordelene ved COCOMO er at:

1. Man tydeligt kan se, hvorfor man får de estimater, man får.
 2. Man i Intermediate COCOMO kan se, hvad det koster at opnå ændringer, ved at se på de nødvendige ændringer i software-relaterede faktorer og overveje, hvad man kan vinde ved en evt. ændring.
-

Basal COCOMO

Den basale COCOMO tager udgangspunkt i et estimat for antallet af tusind linier kildetekst: Kilo Delivered Source Instructions (KDSI). Du kan læse mere om estimering af antallet af kodelinier i afsnittet:

- Estimering af antal linier kode

Ud fra KDSI beregnes indsatsen som projektet kræver (i alt, ikke kun til kodning) i personmåneder (PM). En COCOMO personmåned svarer til 19 effektive arbejdsdage eller 152 effektive timer. Og ud fra antallet af personmåneder beregnes antallet af måneder til udvikling (MU).

Beregningen af PM og MU sker ud fra en formel, hvor tallene i formelen afhænger af, om projektet er organisk, embedded, eller semi-detached. Basal COCOMO tager med andre ord højde for tre forskellige typer projekter.

Basal COCOMO antager, at kravspecifikationen ligger rimelig fast tidligt i projektet, samt at projektet er vel ledet både på udvikler- og kundeside.

Basal COCOMO omfatter langt de fleste aktiviteter i et udviklingsforløb, dog ikke forstudie eller foranalyse ("Feasibility Study"), installation, konvertering og drift.

Basal COCOMO. Formel for organisk udvikling

Organisk udvikling er, når projektet i en relativt lille gruppe udvikler software af en velkendt type til in-house brug. Hovedparten af gruppen har erfaring med tilsvarende systemer i organisationen. Basal COCOMO formelen er så:

$$PM = 2,4 (KDSI)^{1,05} \quad MU = 2,5 (PM)^{0,38}$$

Hvor	PM	=	PersonMåneder
	KDSI	=	Tusind linier kildetekst
	MU	=	Måneder til udvikling

Basal COCOMO. Formel for embedded udvikling

Embedded (indlejret) udvikling er, når projektet f.eks. omfatter ny teknologi, eller algoritmer vi ikke kender godt, eller en ny innovativ udviklingsmetode. Mest karakteristisk for embedded udvikling er, at projektet er underlagt mange bindinger ("tight constraints"). Embedded COCOMO formelen er så:

$$PM = 3,6 (KDSI)^{1,20} \quad MU = 2,5 (PM)^{0,32}$$

Hvor	PM	=	PersonMåneder
	KDSI	=	Tusind linier kildetekst
	MU	=	Måneder til udvikling

Basal COCOMO. Formel for semi-detached udvikling

Semi-detached udvikling er en mellemting mellem organisk og embedded, f.eks. når projektet både har organiske og indlejrede karakteristika. Det kunne f.eks. være et projekt, hvor man skal udvikle software af en velkendt type til in-house brug, men samtidig for første gang skal bruge et nyt CASE-værktøj, og derfor oven i købet har tilknyttet mange projektdeltagere på mindre end fuld tid, som skal "snuse" til brugen af CASE-værktøjet. Semi-detached COCOMO formelen er:

$$PM = 3,0 (KDSI)^{1,12} \quad MU = 2,5 (PM)^{0,35}$$

Hvor	PM	=	PersonMåneder
	KDSI	=	Tusind linier kildetekst
	MU	=	Måneder til udvikling

Eksempel på brug af basal COCOMO formel til semi-detached udvikling

Vi antager, at vi har estimeret et projekt til at omfatte 10.000 linier kildetekst. Udviklingstypen antages at være semi-detached.

$$PM = 3,0 (10)^{1,12} = 40 \text{ personmåneder} = 6080 \text{ effektive arbejdstimer}$$

$$MU = 2,5 (40)^{0,35} = 9,1 \text{ måneder til udvikling}$$

$$\text{Gennemsnitlig bemanning: } 40 / 9,1 = 4,4 \text{ mand}$$

Vi kan med andre ord gå til ledelsen og styregruppen og sige, at projektet kan gennemføres på ca. 6000 arbejdstimer. Med jævn bemanning gennem hele projektforløbet vil det f.eks. sige fem mand i næsten fuld tid i lidt mere end ni måneder fra den dag de starter.

Intermediate COCOMO

Intermediate COCOMO tager højde for 15 faktorer omhandlende dels den software, der skal udvikles, dels krav til hardware, dels personlige faktorer – hvem er det, der udvikler, samt nogle projektspecifikke faktorer. Igen skelnes der mellem tre typer udvikling – organisk, embedded og semi-detached, og faktorerne ganges så efter nedenstående princip:

Organisk	$PM = 3,2 (KDSI)^{1,05} * f_1 * f_2 * f_3 \dots f_{14} * f_{15}$
Semi-detached	$PM = 3,0 (KDSI)^{1,12} * f_1 * f_2 * f_3 \dots f_{14} * f_{15}$
Embedded	$PM = 2,8 (KDSI)^{1,20} * f_1 * f_2 * f_3 \dots f_{14} * f_{15}$

Ud fra det beregnede antal personmåneder beregnes antal måneder til udvikling (MU) med samme formler som ved basal COCOMO:

Organisk $MU = 2,5 (PM)^{0,38}$

Semi-detached $MU = 2,5 (PM)^{0,35}$

Embedded $MU = 2,5 (PM)^{0,32}$

Nedenfor findes en oversigt over alle de femten faktorer, der ganges med i Intermediate COCOMO.

Software-relaterede faktorer der ganges på i Intermediate COCOMO

f1 = Pålidelighed krævet af systemet (RELY)

0,75 = Manglende pålidelighed fører kun til meget lille forstyrrelse

0,88 = Manglende pålidelighed har kun lille betydning

Det er let at genskabe tab

1,00 = Manglende pålidelighed har moderat betydning

Det er muligt at genskabe det tabte

1,15 = Manglende pålidelighed har stor betydning og kan medføre store økonomiske tab

1,40 = Manglende pålidelighed har stor betydning og kan i værste fald medføre tab af menneskeliv

f2 = Størrelsen af databasen relativt til programstørrelsen (DATA)

Database-størrelse angives i Bytes (estimat)

Programstørrelse angives i linier kildetekst

0,94 = Database-størrelse / programstørrelse < 10

1,00 = $10 \leq \text{Database-størrelse} / \text{programstørrelse} < 100$

1,08 = $100 \leq \text{Database-størrelse} / \text{programstørrelse} < 1000$

1,16 = Database-størrelse / programstørrelse ≥ 1000

f3 = Komplexitet af systemet (CPLX). Vælg den af nedenstående grupper der bedst beskriver kompleksiteten

0,70 = Kontroloperationer: Lige ud af landevejen kode med meget få indlejrede DO-, CASE- og IF-sætninger

Beregninger: Evaluering af simple udtryk af typen $A = (B+C) * (D-E)$

Ydre enheds operationer: Simple READ og WRITE sætninger

Databehandling: Simple ARRAY i hukommelsen

0,85 = Kontroloperationer: Ligeftem implementering af struktureret programmering. For det meste simple erklæringer

Beregninger: Evaluering af lettere udtryk af typen $A = \sqrt{B^{2,4} * C * D}$

Ydre enhedsoperationer: Ikke nødvendigt at kende specifik enhed. Simple GET og PUT operationer

Databehandling: Enkle filer uden ændringer af datastrukturen. Ingen ændringer af filindhold. Ingen midlertidige filer

1,00 = Kontroloperationer: For det meste simple indlejringer. Nogle kontroloperationer mellem moduler. Beslutningstabeller

Beregninger: Brug af standard matematik- og statistik-rutiner. Basale vektor og matrix-operationer

Ydre enhedsoperationer: Input/output behandling omfatter valg af ydre enhed, check af status, og fejlhåndtering

Databehandling: Input fra mange filer. Output til en fil. Simple ændringer af filstruktur, simple ændringer i filer

1,15 = Kontroloperationer: Komplekse indlejringer med mange betingede erklæringer. Håndtering af køer og stakke. Mange kontroloperationer mellem moduler

Beregninger: Basal numerisk analyse og differential-ligninger. Basale afrundinger og afskæringer

- Ydre enhedsoperationer: Operationer på det fysiske niveau i relation til input/output (fysiske lageradresser, læsninger, søgninger etc.)
 Databehandling: Subrutiner til specielle formålaktiveret på grundlag af indholdet af data. Komplex restrukturering af data på RECORD niveau
- 1,30 = Kontroloperationer: Rekursion i programmer. Håndtering af på forhånd fastlagte afbrydelser (interrupts)
 Beregninger: Svær men struktureret numerisk analyse. Matrixregning. Partiel differentiering
 Ydre enhedsoperationer: Rutiner til diagnose og håndtering af afbrydelser (interrupts). Håndtering af kommunikationslinier
 Databehandling: En generel parameterdrevet filstruktureringsrutine Skabelse af nye filer. Fortolkning af kommandoer. Optimering af søgninger
- 1,65 = Kontroloperationer: Multipel ressourcehåndtering og tildeling ("Scheduling") med dynamiske ændringer af prioriteringer. Kontroloperationer på mikrokodeniveau
 Beregninger: Svær og ustruktureret numerisk analyse. Analyse af stokastiske data med stor præcision
 Ydre enhedsoperationer: Realtids-afhængig kodning i forhold til den specifikke ydre enhed.
 Mikrokode operationer
 Databehandling: Stærkt koblede, dynamiske, relationelle strukturer. Håndtering af naturligt sprog
-

Computer-relaterede faktorer der ganges på i Intermediate COCOMO

f4 = Krav til udførelsestid i drift (TIME)

- 1,00 = mindre end 50% brug af til rådighed værende udførelsestid
 1,11 = maksimalt 70% brug af til rådighed værende udførelsestid
 1,30 = maksimalt 85% brug af til rådighed værende udførelsestid
 1,66 = maksimalt 95% brug af til rådighed værende udførelsestid

f5 = Krav til hovedlager ("Main storage") i drift (STOR)

- 1,00 = mindre end 50% brug af til rådighed værende lager
 1,11 = maksimalt 70% brug af til rådighed værende lager
 1,30 = maksimalt 85% brug af til rådighed værende lager
 1,66 = maksimalt 95% brug af til rådighed værende lager

f6 = Hyppighed af ændringer i teknisk platform (VIRT)

- 0,87 = Større ændring hver 12. måned. Mindre ændring hver måned
 1,00 = Større ændring hver 6. måned. Mindre ændring hver 2. uge
 1,15 = Større ændring hver 2. måned. Mindre ændring hver uge
 1,30 = Større ændring hver 2. uge. Mindre ændring hver 2. dag

f7 = Ventetid ved testkørsler på at få resultater (TURN)

- 0,87 = Interaktivt
 1,00 = Mindre end 4 timer
 1,07 = Fra 4 til 12 timer
 1,15 = Mere end 12 timer
-

Person-relaterede faktorer der ganges på i Intermediate COCOMO

f8 = Analytikernes kapacitet / evne (ACAP). Vælgdet udsagn fra nedenstående grupper, der bedst beskriver den typiske analytiker i projektet.

- 1,46 = Blandt de 85% bedste analytikere
 1,19 = Blandt de 65% bedste analytikere
 1,00 = Blandt de 45% bedste analytikere
 0,86 = Blandt de 25% bedste analytikere
 0,71 = Blandt de 10% bedste analytikere

f9 = Udviklernes erfaring i applikationsdomæne (AEXP)

- 1,29 = Mindre end 4 måneder
- 1,13 = 1 år
- 1,00 = 3 år
- 0,91 = 6 år
- 0,82 = 12 år

f10 = Programmørernes kapacitet / evne (PCAP). Vælg det udsagn fra nederstående grupper, der bedst beskriver den typiske programmør i projektet.

- 1,42 = Blandt de 85% bedste programmører
- 1,17 = Blandt de 65% bedste programmører
- 1,00 = Blandt de 45% bedste programmører
- 0,86 = Blandt de 25% bedste programmører
- 0,70 = Blandt de 10% bedste programmører

f11 = Udviklernes erfaring med teknisk platform (VEXP)

- 1,21 = Mindre end 1 måned
- 1,10 = 4 måneder
- 1,00 = 1 år
- 0,90 = 3 år

f12 = Programmørernes erfaring med programmeringssprog (LEXP)

- 1,14 = Mindre end 1 måned
- 1,07 = 4 måneder
- 1,00 = 1 år
- 0,95 = 3 år

Projekt-relaterede faktorer der ganges på i Intermediate COCOMO

f13 = Graden af anvendelse af moderne programmering (MODP)

- 1,24 = Ingen brug
- 1,10 = Begyndende brug
- 1,00 = Nogen brug
- 0,91 = Bruges normalt
- 0,82 = Rutinemæssig brug

f14 = Brug af programmeringsværktøjer (TOOL)

- 1,24 = Basale mikro-værktøjer (assembler-niveau)
- 1,10 = Basale mini-værktøjer
- 1,00 = Basale midi- og maxi-værktøjer
- 0,91 = Stærke maxi-programmerings-værktøjer. Test-værktøjer
- 0,83 = Desuden krav-, design, og projektledelsesværktøjer

f15 = Krav til tidsplan / leveringstidspunkt (SCED)

Den nominelle udviklingstid beregnes vha. af formlerne:

- | | |
|---------------|------------------------|
| Organisk | $MU = 2,5 (PM)^{0,38}$ |
| Semi-detached | $MU = 2,5 (PM)^{0,35}$ |
| Embedded | $MU = 2,5 (PM)^{0,32}$ |
- 1,23 = Krav om gennemførelse på mindre end 75% af den nominelle tid
 - 1,08 = Krav om gennemførelse på mindre end 85% af den nominelle tid
 - 1,00 = Krav om gennemførelse på nominel tid
 - 1,04 = Krav om gennemførelse på mere end 130% af den nominelle tid
 - 1,10 = Krav om gennemførelse på 160% eller mere ift. den nominelle tid

Er COCOMO uanvendelig tyve år efter ?

Den største svaghed ved COCOMO er selvfølgelig, at udgangspunktet for alle beregninger er antallet af kodelinier. Og selv om rutinerede programmører ofte er temmelig gode til at estimere antallet af kodelinier i et modul, der er beskrevet, så vil man ofte være langt henne i projektet – typisk i designfasen - før man kan beskrive modulet tilstrækkeligt til at estimeringen kan foregå.

COCOMO er med andre ord ikke til megen hjælp i starten af et projekt, hvor man har allermest brug for at få en idé om størrelsesordenen af projektet. Det er også ca. 20 år siden, at Barry Boehm indsamlede tallene fra de projekter som han brugte til at etablere de tal, der indgår i både basal og intermediate COCOMO. Siden da har den teknologiske udvikling bestemt ikke ligget stille, så man kan nemt forestille sig, at de multiplikatorer der f.eks. indgår i intermediate COCOMO for en stor del er forældede. Tænk blot på:

- Krav til udførelsestid i drift (TIME), samt krav til hovedlager (“Main storage”) i drift (STOR), der begge bygger på det faktum, at for 20 år siden var disse faktorer knappe ressourcer, mens det forekommer langt mere sjældent i dag
 - Ventetid ved testkørsler for at få resultater (TURN), hvor det idag er næsten utænkeligt ikke at have et interaktivt testværktøj
 - Graden af anvendelse af moderne programmering (MODP), som idag typisk burde inddrage graden af anvendelse af visuel programmering
 - Brug af programmeringsværktøjer (TOOL), hvor de værktøjsniveauer der nævnes (mikro, mini, midi, maxi) er nærmest uforståelige i dag. Her skulle snarere indgå noget om CASE-værktøjer
-

COCOMO 2.0

Version 2.0 af COCOMO blev udviklet i midten af 90'erne

For at opdatere COCOMO og derved bl.a. imødegå påstande om, at COCOMO var forældet og uanvendelig, begyndte et konsortium med bl.a. Barry Boehm udviklingen af COCOMO version 2.0 i midten af 1990'erne (Sutzke, 1997).

Den væsentligste ændring i COCOMO 2.0 er, at man nu har taget højde for, at det er svært at estimere antallet af kodelinier tidligt i et projekt. Derfor har man til administrative systemer udviklet en metode kaldet objekt point. Disse objekt point kan tælles allerede under foranalysen. I kravspecifikationsfasen kan man i COCOMO 2.0 anvende Function Points, som beskrevet af IFPUG i 1994.

Endelig kan man, når man har påbegyndt designet af sit program som hidtil, tage udgangspunkt i et estimat af antal linier kode (som i den oprindelige COCOMO)

Ny basal COCOMO 2.0

COCOMO 2.0 har opgivet den oprindelige skelnen mellem udviklingstyperne organisk, embedded og semidetached. I stedet anvendes nu formlen:

$$PM_{\text{nominel}} = 3,0 (KDSI)^{1,01 + B}$$

til at beregne det første bud på hvor mange personmåneder, der vil medgå til udviklingen. I formlen indgår "B" som sammensættes ved at se på fem projektspecifikke forhold, der hver især vurderes fra meget lav (5 point), over lav (4 point), almindelig (3 point), høj (2 point), meget høj (1 point) og ekstra høj (0 point). De fem forhold er:

1. Familiaritet ("precedentedness"). Helt igennem familiært projekt = 0, næsten helt familiært = 1, generelt familiært = 2, noget ukendt = 3, næsten ukendt = 4, helt igennem nyt = 5.
 2. Udviklingsfleksibilitet – Hvor fleksible er kravene. Der findes kun nogle generelle mål at udvikle efter = 0, der er krav om nogen konformitet = 1, mere konformitet = 2, almindeligt fleksibelt projekt = 3, megen præcision = 4, kravene skal efterleves helt rigtigt og strengt ("rigorous") = 5.
 3. Arkitektur/risikohåndtering. Procentdel af grænseflader mellem moduler der er specificeret og procentdel af væsentlige risici der er elimineret. Fuldt ud 100% = 0, for det meste 90% = 1, generelt 75% = 2, ofte 60% = 3, noget 40% = 4, lidt 20% = 5.
 4. Team Spirit. Helt problemfri ("seamless") interaktion = 0, meget samarbejdende = 1, for det meste samarbejdende = 2, generelt samvirke = 3, nogle vanskelige interaktioner = 4, meget besværlig interaktion = 5.
 5. Procentdel af de 18 nøgleområder ("Key Process Areas") i CMM der efterleves i organisationen. Fuld opfyldelse = 0, ingen opfyldelse = 5. For en nærmere forklaring af hvad CMM omfatter se afsnittet:
 - Modenhed og estimering
-

Eksempel på brug af basal COCOMO 2.0 formelen

Vi antager, at vi har estimeret et projekt til at omfatte 10.000 linier kildetekst.

For de fem projektspecifikke forhold gælder, at:

1. Vi er generelt familiære med det programmel, der skal udvikles = 2.
2. Projektet udvikles efter nogle meget generelt definerede mål = 0.
3. Grænseflader og risici er stort set ikke behandlet = 5.
4. Vores udviklingsteam har tidligere haft noget vanskeligt ved interaktion, dvs. en del skænderier internt = 4.
5. Vi opfylder ingen af de 18 nøgleområder fra CMM = 5.

$$\begin{aligned} B &= 0,02 \text{ (familiaritet)} + 0,00 \text{ (udviklingsfleksibilitet)} + 0,05 \text{ (grænseflader)} + 0,04 \text{ (team)} + 0,05 \\ &\quad \text{(modenhed)} \\ &= 0,16 \end{aligned}$$

$$\begin{aligned} PM_{\text{nominel}} &= 3,0 (10)^{1,01 + 0,16} \\ &= 44,4 \text{ personmåneder} \end{aligned}$$

Faktorbaseret COCOMO 2.0

Den gamle intermediate COCOMO er blevet opdateret, dels med nye faktorer, dels med nye forklaringer, og dels med ændrede multiplikatorer. Som hovedregel har man dog forsøgt at bevare så meget af den gamle intermediate COCOMO som mulig.

Ud fra det beregnede antal nominelle personmåneder beregnes et justeret antal personmåneder ud fra formlen:

$$PM_{\text{justeret}} = PM_{\text{nominel}} * f1 * f2 * f3 \dots f16 * f17$$

Nedenfor findes en kort gennemgang af de sytten faktorer i COCOMO 2.0, sammenholdt med den oprindelige intermediate COCOMO. Beskrivelsen af de oprindelige faktorer i intermediate COCOMO findes i afsnittet om:

- COCOMO

Væsentlige ændringer i version 2.0 i forhold til den oprindelige COCOMO er skrevet med kursiv.

- f1 = Pålidelighed krævet af systemet (RELY)
 - f2 = Størrelsen af databasen (DATA)
 - f3 = Komplexitet af systemet (CPLX). *Få ændringer af forklaringer. Ny brugergrænseflade-dimension inddraget*
 - f4 = *Krævet grad af genbrug (RUSE)*
 - f5 = *Krævet dokumentation (DOCU)*
 - f6 = Krav til udførelsestid i drift (TIME)
 - f7 = Krav til hovedlager ("Main storage") i drift (STOR)
 - gl. f7 = Ventetid ved testkørsler på at få resultater (TURN). *Droppet*
 - f8 = Hyppighed af ændringer i teknisk platform (VIRT *ændret til PVOL*)
 - f9 = Analytikernes kapabilitet / evne (ACAP). *Multiplikator øget efter at empiri har påvist større betydning*
 - f10 = Udviklernes erfaring i applikationsdomæne (AEXP)
 - f11 = Programmørernes kapabilitet / evne (PCAP)
 - f12 = Udviklernes erfaring med teknisk platform (VEXP). *Multiplikator øget*
 - f13 = Programmørernes erfaring med prog.sprog og udviklingsværktøjer (LEXP)
 - gl. f13 = Graden af anvendelse af moderne programmering (MODP). *Droppet*
 - f14 = *Kontinuitet i personale. Måles i årlig udskiftning. (PCON)*
 - f15 = Brug af programmeringsværktøjer (TOOL)
 - f16 = *Graden af udvikling på flere lokaliteter samt kommunikationsfaciliteter imellem lokaliteter (SITE)*
 - f17 = Krav til tidsplan / leveringstidspunkt (SCED)
-

Nye faktorer i COCOMO 2.0

Kort kan man sige, at COCOMO 2.0 i forhold til den oprindelige COCOMO har ændret sig på ni punkter:

- n1 = Krævet dokumentation
 - n2 = Grad af krævet genbrug
 - n3 = Udviklerkontinuitet
 - n4 = Udvikling spredt på flere lokationer
 - n5 = Organisationens erfaring med projekttype
 - n6 = Kravenes fleksibilitet
 - n7 = Opgavens velafklarethed, specielt grænseflader og risici
 - n8 = Team Spirit
 - n9 = Den faglige modenhed i udviklingsorganisation
-

Estimering af brugergrænsefladen

Prototyper af brugergrænseflader vinder frem

I takt med udbredelsen af grafiske brugergrænseflader ("GUI"), f.eks. på PC'ere, er det blevet mere og mere populært at lave prototyper af brugergrænsefladen.

McConnell (1998) har skitseret en udviklingsmodel, hvor man bruger en prototype som den centrale del af specifikationen af krav, frem for at skrive en traditionel kravspecifikation:

1. Identificér central gruppe af slutbrugere.
2. Interview slutbrugerne.
3. Byg en simpel brugergrænseflade prototype.
4. Skriv en "style guide" ud fra prototypen, når slutbrugerne er enige om den. "Style guiden" skal kun være på få sider - nok til at lave en fuld prototype.
5. Færdiggør en fuld prototype (stadig beregnet på at smide væk efter brug) og brug den som baseline specifikation.
6. Skriv detaljeret slutbruger-dokumentation baseret på prototypen.
7. Lav et separat kravdokument for ikke-brugergrænseflade områder, f.eks. med ikke-funktionelle krav og tekniske grænseflader til andre systemer.

"The approach described ... works best for user intensive, small to medium-sized projects" siger Steve McConnell (1998: side123).

Det er klart, at hvis en sådan arbejdsform vinder frem-hvilket meget tyder på - så vil den færdige prototype (punkt 5. oven for) være det grundlag, som man har at estimere ud fra tidligt i et projekt.

GUI-baseret estimering

Roetzheim & Beasley (1998) gør rede for, hvilke særlige ting man kan tælle i en brugergrænseflade. Konkret foreslår de at tælle:

1. **Dialogbokse**, både til dataindtastning og til valg i f.eks. en palette. Simple dialogbokse, der blot indeholder en meddelelse man kan klikke OK til, tælles ikke.
2. Mulige **menuvalg** tælles. Menuer, der blot giver adgang til undermenuer, tælles ikke. Menuvalg, der optræder flere steder, tælle kun én gang.
3. **Rapporter**. Hver samling af data, der rapporteres, tælles som en.
4. **Tabeller** inkluderer antallet af logiske relationelle tabeller i systemet.
5. **Vinduer** omfatter antallet af uafhængige vinduer i systemet.

Man kan ikke alene ud fra disse fem ting i en brugergrænseflade estimere størrelsen eller indsatsen, som et givent projekt kræver. Man er selvfølgelig nødt til at supplere med nogle overvejelser om funktionalitet.

Den beskrevne måde at tælle på indgår i et værktøj, der er en del af Roetzheim & Beasley (1998). En næsten tilsvarende måde at tælle på indgår i teknikken kaldet Objekt Point.

I COCOMO-modellen er brugergrænseflade-dimensionen i den nyeste version blevet inddraget som en del af kompleksitetsfaktoren.

I den nyeste udgave af reglerne fra IFPUG ("International Function Point Users Group") om hvordan man tæller Function Points, har man ændret reglerne for tælling af uddata, så de mange fejlmeddelelser i moderne brugergrænseflader om alt mellem himmel og jord ikke tæller med samme vægt som andre uddata. Desuden er reglerne for tælling generelt blevet modificeret, således at tallene der tælles passer bedre til Client/Server og grafiske brugergrænseflader.

Objekt point

Moderne applikationsudvikling med CASE-værktøjer og prototyper

I forbindelse med moderne administrativ systemudvikling foregår mere og mere af arbejdet i integrerede CASE-værktøjer (såkaldt ICASE) med visuelle faciliteter, så man kan "tegne" sit program, og med automatisk kodegenerering, samt mulighed for at lave hurtige prototyper.

I den forbindelse har folkene bag COCOMO 2.0 (Boehm et al. 1995) udviklet de såkaldte objekt point. Idéen er, at man i de fleste CASE-værktøjer kan tale om objekter i form af enten skærbilleder, rapporter eller traditionelle programmoduler.

Seks trin fra objekt point til estimat

For at nå frem til et estimat gennemløber man seks trin.

47. Vurdér antallet af objekter for hver af de tre typer: skærbilleder, rapporter og traditionelle programmoduler.
48. Klassificér hvert eneste skærm- og rapport-objekt som simpelt, medium eller svært ved hjælp af nedenstående tabeller:

SKÆRM-OBJEKTER			
Antal og kilde for data-tabeller: Antal views omfattet af skærbillede:	Mindre end 4, heraf højst 2 fra server, og højst 3 fra klient	Mindre end 8, heraf højst 3 fra server, og højst 5 fra klient	Mere end 8, eller mere end 3 fra server el. > 5 fra klient
Mindre end 3	simpelt vægt = 1	simpelt vægt = 1	medium vægt = 2
Mellem 3 og 7	simpelt vægt = 1	medium vægt = 2	svært vægt = 3
Mere end 8	medium vægt = 2	svært vægt = 3	svært vægt = 3

RAPPORT-OBJEKTER			
Antal og kilde for data-tabeller: Antal sektioner (dele) omfattet af rapport:	Mindre end 4, heraf højst 2 fra server, og højst 3 fra klient	Mindre end 8, heraf højst 3 fra server, og højst 5 fra klient	Mere end 8, eller mere end 3 fra server el. > 5 fra klient
Mindre end 3	simpelt vægt = 2	simpelt vægt = 2	medium vægt = 5
Mellem 3 og 7	simpelt vægt = 2	medium vægt = 5	svært vægt = 8
Mere end 8	medium vægt = 5	svært vægt = 8	svært vægt = 8

49. Tæl nu vægtene sammen for hvert eneste skærm- og rapport-objekt. Tildel desuden alle traditionelle programmeringsmodul-objekter vægten 10. Herved fremkommer VOP (Vægtede Objekt Point).
50. Tag højde for genbrug ved hjælp af formlen:

$NOP (\text{Nye Objekt Point}) = (\text{vægtede objekt point}) (100 - \% \text{ genbrug}) / 100$

51. Bestem produktiviteten ud fra nedenstående tabel. Udviklernes erfaring sættes ind. ICASE modenhed sættes ind. Et passende gennemsnit aflæses.

Udviklernes erfaring	Meget lav	Lav	Typisk	Stor	Meget stor
ICASE modenhed	Meget lav	Lav	Typisk	Stor	Meget stor
PRODuktivitet	4	7	13	25	50

52. Beregn antal personmåneder (PM) til udvikling ved hjælp af formlen:

$PM = NOP / PROD$ (fra tabellen)

Eksempel på beregning af Objekt Point

Vi befinder os i en stor administrativ IT-organisation med hundredvis af udviklere og flerårig erfaring med ICASE.

Vi er blevet bedt om at lave et nyt timeregistreringssystem, så vi bedre kan fastholde erfaringer fra projekter. En prototype er blevet udviklet og afprøvet. Ud fra prototypen har vi skrevet en style guide for applikationen. Guiden indgår i kravspecifikationen.

Ud fra kravspecifikationen kan vi se, at der skal laves 3 rapporter, dels for enkeltpersoner, dels for afdelinger, dels for projekter. Hver af de tre rapporter skal have fat i person-tabellen, projekt-tabellen og timerapport-tabellen, der alle ligger placeret på klienter. Desuden skal projektrapporten også have fat i firmaets samlede erfaringsdata-tabel, samt i tabellen med projektplandata, der begge ligger centralt. I tabellen er vi altså i kolonnerne "Mindre end 4" for de to rapporter, og i "Mindre end 8" for projektrapporten.

Den enkelte persons rapport og afdelingsrapporten indeholder kun en sektion, så disse to rapporter kan vi i tabellen aflæse til at være simple. Derimod omfatter projektrapporten fire sektioner, dels med individer på projektet, dels med aggregerede projektdata fordelt på faser, dels med faktiske data sammenholdt med erfaringsdata, og endelig med faktiske data sammenholdt med planen. Denne sidste rapport kan altså aflæses til at være svær.

Ud fra prototypen og kravspecifikationen kan vi se, at der skal indgå 8 skærbilleder i systemet. To af dem har fat i flere end 4 tabeller, dvs. at de er af typen medium. De seks andre er simple.

En simpel nedbrydning af den ønskede funktionalitet fører til, at vi skal skrive fem modul-objekter i ICASE-værktøjet.

I alt ser beregningen således ud:

2 simple rapporter	2 * 2 point
1 svær rapport	8 point
6 simple skærbilleder	6 * 1 point
2 medium skærbilleder	2 * 2 point
5 modul-objekter	5 * 10 point
	72 objekt point

Der er ingen genbrug i projektet så : $VOP = NOP$.

Udviklernes erfaring er typisk ("Nominal"), mens vores anvendelse af ICASE-værktøjet er blandt det mest modne, man kan finde i verden (vi har tit besøg fra udlandet for at de kan se, hvordan vi gør). Vi vil kategorisere ICASE modenheden som meget høj. Tilsammen vælger vi at tage PROD-faktoren fra kolonnen høj, dvs. 25.

Nu kan vi så beregne antallet af personmåneder, der vil medgå til projektet:

$$PM = NOP / PROD = 72 / 25 = 2,88$$

Vi skal altså afsætte omkring tre personmåneder til projektet med at udvikle timeregisteringssystemet.

Function Points

Historien om Function Points startede i 1979

I oktober 1979 publicerede A.J.Albrecht fra IBM en funktionsbaseret metrik, hvor man ved at tælle en række synlige aspekter ved software, som samtidig er aspekter der er vigtige for brugerne, kan få et mål for størrelsen af software.

Albrechts oprindelige artikel blev publiceret bredt over hele verden i 1981. En revideret version blev publiceret i 1984 af Albrecht og IBM. Revisionen tager højde for kompleksitet på en systematisk måde, og revisionen definerede fjorten faktorer der skulle anvendes til at justere det "rå" antal Function Points. Den reviderede version dannede også udgangspunkt for International Function Point Users Group's (IFPUG) første tælleregler. IFPUG eksisterer fortsat som en uafhængig non-profit organisation med hovedkvarter i Ohio, USA.

I 1985 introducerede Software Productivity Research (SPR), bl.a. med Capers Jones, en simplere måde at tage højde for kompleksitet på end IBM's metode. Samme SPR introducerede i 1986 Feature Points - en slags udvidede Function Points specielt beregnet til realtids- og systemsoftware.

I 1990 udgav IFPUG en manual med standard tælleregler for Function Points, som blev meget udbredt. I 1994 kom en opdateret såkaldt "Release 4.0 of Function Point Counting Practices Manual", hvor især fejlmeddelelser tælles anderledes end tidligere.

Grundlæggende er der fem ting, man tæller

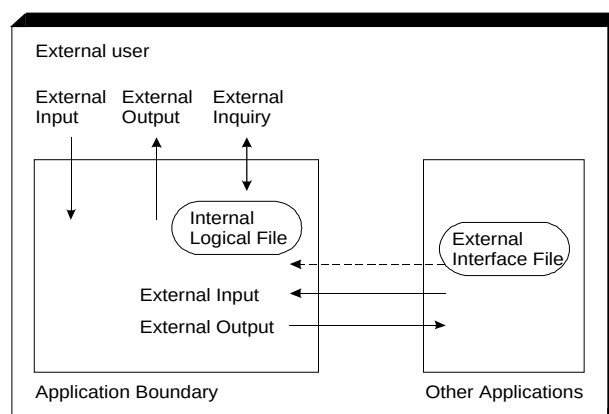
"External Input" (EI) er skærbilleder eller blanketter, hvor brugeren eller et andet program indtaster nye data eller opdaterer eksisterende data. Hvis et indtastings-skærbillede er for stort til at stå på en skærm, så tælles det alligevel kun for ét eksternt input. Man tæller med andre ord input, der kræver separat databehandling, her et logisk skærbillede frem for et fysisk.

"External Output" (EO) er skærbilleder eller rapporter, som applikationen producerer til en bruger eller andre programmer. Det man tæller er uddata, der kræver separat databehandling. Et lagerstyringsprogram, der f.eks. udskriver 100 lageropgørelser, tæller kun som et EO.

"External Inquiry" (EQ) er skærbilleder der tillader brugeren at interagere med applikationen, søge efter ting, eller forespørge assistance eller information. Et hjælpe-skærbillede vil tælle som et EQ.

"Internal Logical File" (ILF) er logiske samlinger af kartoteksblade ("records"), som applikationen ændrer eller opdaterer. ILF kan være en flad fil, eller det kan være en tabel i en SQL-database.

"External Interface File" (EIF) er filer der deles med andre programmer, såsom delte databaser.



Eksempel på simpel Function Point beregning

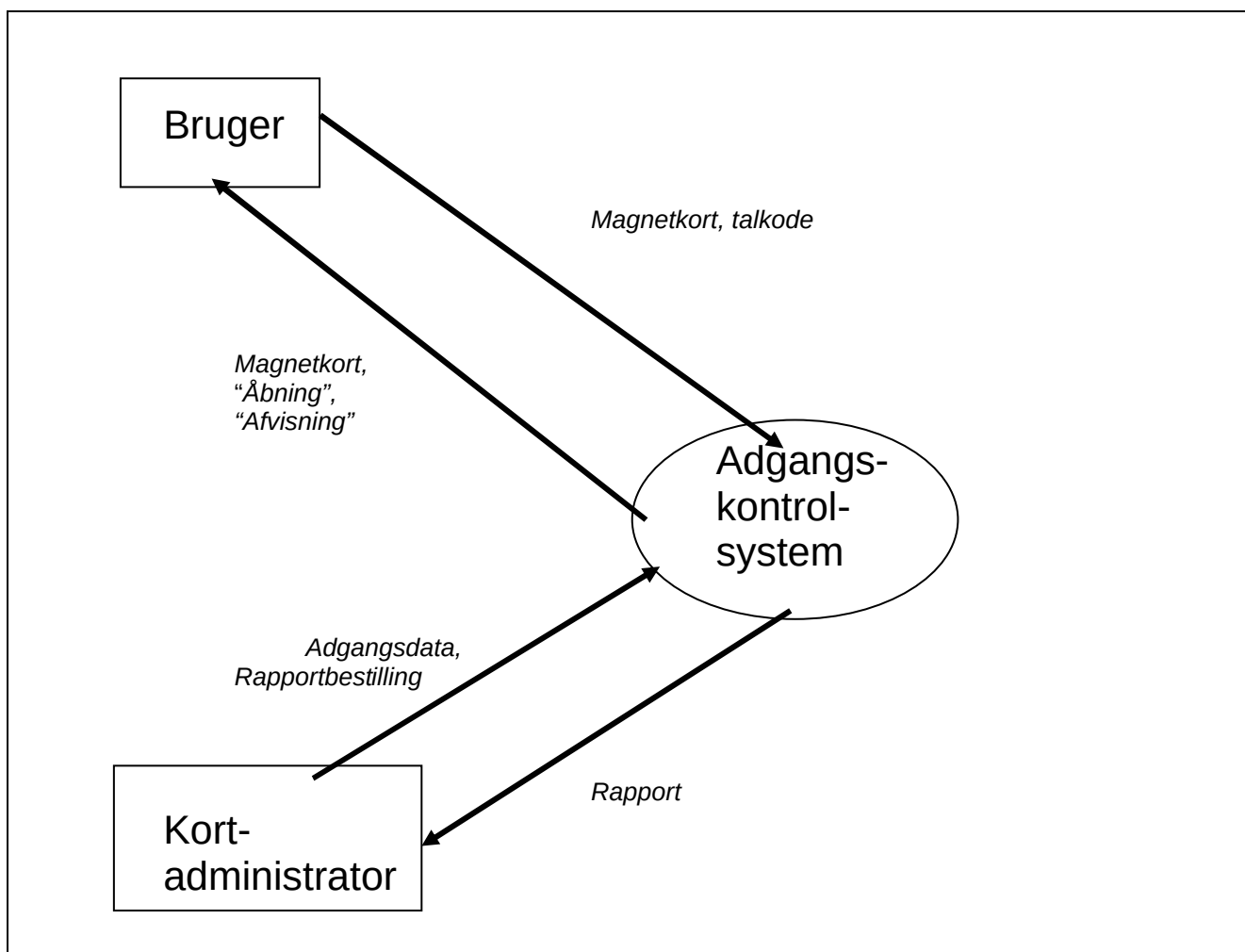
Lad os forestille os følgende situation: Hos FOXTROT har man erkendt, at man bruger alt for mange ressourcer på adgangskontrol. Man hverken kan eller vil dog undvære kontrollen, idet man har ansvar for en række meget fortrolige oplysninger, som nødtigt skulle falde i de forkerte hænder. Efter at have analyseret sagen grundigt, har man besluttet at anskaffe et elektronisk system til adgangskontrol, hvor hver bruger får et magnetkort og en 4-cifret talkode.

Den grundlæggende funktion i adgangskontrol-systemet hos FOXTROT skal være at tillade eller afvise adgang baseret på en sammenligning af magnetkort og indtastet talkode med adgangstilladelser registreret i en database. Denne primære funktion medfører et behov for en række sekundære funktioner, såsom:

- Udstedelse af et magnetkort
- Tildeling og/eller ændring af en talkode til et magnetkort
- Registrering og/eller ændring af en adgangstilladelse

På figuren nedenfor er vist et kontekstdiagram for adgangskontrolsystemet, hvoraf det ses, at der er to eksterne kilder til data, dels en bruger, dvs. en ansat med behov for adgang det pågældende sted, dels en kortadministrator.

Brugeren kommer med sit magnetkort og en talkode for at få adgang. Dette er illustreret med pilen fra "Bruger" ind til systemet.



Magnetkort eller talkode kan blive afvist af systemet. Eller Brugeren kan få sit magnetkort tilbage samtidig med, at der åbnes til det pågældende sted. Dette er illustreret med pilen fra systemet til Brugeren".

Kartoteks-administratoren er den person, der dels holder styr på, hvem der har adgang til hvad hvornår, f.eks. hvilke ansatte der har hvilke magnetkort, dels sørger for at få udskrevet adgangssporter, hvis og når man har brug for at kontrollere, hvem der har været hvor hvornår.

En gruppe designere fra FOXTROT har vurderet følgende med hensyn til de fem grundlæggende ting der tælles:

"External Input" (EI). Systemet vil indeholde fire skærbilleder, et til udstedelse af et magnetkort, et til tildeling og/eller ændring af en talkode til et magnetkort, et til registrering og/eller ændring af en adgangstilladelse, samt et skærbillede til bestilling af rapporter. "External Output" (EO). Systemet vil indeholde to EO. Et der omhandler "afvist" eller "åbning" over for brugeren, samt en rapport til kartoteksadministratoren. "External Inquiry" (EQ). Vi vurderer, at der kun er brug for to meget simple typer forespørgsler, dels om en person har adgang et bestemt sted, dels om en person har fået adgang et givent sted inden for en given periode. "Internal Logical File" (ILF). I vores system er der to grundlæggende entiteter; magnetkort og adgangssteder. Mellem disse entiteter er der en mange-til-mange relation. Vi indfører derfor en hjælpe-entitet kaldet adgangstilladelse. Relationen mellem magnetkort og de ansatte, der skal have adgang, er lidt mere kompliceret: • Enhver ansat er altid knyttet til en og kun en afdeling, mens en afdeling kan have mange ansatte (en-til-mange relation) • En ansat kan have mange magnetkort, og et magnetkort kan kun høre til én bestemt ansat (en-til-mange relation) • Et magnetkort kan også være knyttet til en bestemt jobfunktion. I så fald kan der kun høre ét magnetkort til jobfunktionen (en-til-en relation) • Et magnetkort kan endelig være knyttet til et bestemt projekt, og projektet kan godt have mange magnetkort tilknyttet • En job-funktion kan høre til en afdeling, men behøver ikke at gøre det. En afdeling kan have en eller flere jobfunktioner tilknyttet Alt i alt ender vi med seks ILF, svarende til en implementering med seks tabeller i en database: Magnetkort, adgangssted, adgangstilladelse, afdeling, jobfunktion, og projekt. Tabellen med ansatte betragter vi som en fil, der deles med andre programmer. "External Interface File" (EIF) er filer, der deles med andre programmer, såsom delte databaser. Dem har vi kun en af, nemlig "ansat".

Nedenfor er i en tabel vist, hvordan ovenstående parametre kan bruges til at regne ud, at størrelsen af adgangskontrol systemet er 101 ikke-justerede Function Points.

Parameter	Antal		Vægt		Total
EI - Eksterne inddata	4	x	4	=	16
EO - Eksterne uddata	2	x	5	=	10
EQ - Eksterne forespørgsler	2	x	4	=	8
ILF - Interne logiske filer	6	x	10	=	60
EIF - Eksterne græseflader	1	x	7	=	7
Ikke-justeret sum					101

Som vægte i tabellen er valgt de tal, der bruges for medium kompleksitet. Hvis IFPUG's standardiserede tælle-måde fra 1990 (der svarer til IBM 1984) skulle følges fuldt ud, så skulle vi for hver eneste af de fundne EI, EO, EQ, ILF og EIF have vurderet, om der var tale om lav, medium eller høj kompleksitet.

Function Point justering

Det beregnede antal function points skal nu justeres, før vi har det endelige tal. I den oprindelige Albrecht 1979 model kunne man efter skøn tillægge eller fratrække op til 25%. I senere modeller er justeringen blevet meget mere systematisk, og man kan nu justere op eller ned med mere end en faktor 2. Her vil de 14 faktorer, som IFPUG 1990 og IBM 1984 anvender, blive gennemgået.

Hver af de fjorten faktorer vægtes efter følgende skala:

- 0 = Faktoren ikke til stede i systemet eller uden betydning.
- 1 = Lille indflydelse.
- 2 = Moderat indflydelse.
- 3 = Gennemsnitlig indflydelse.
- 4 = Signifikant indflydelse.
- 5 = Stærk indflydelse.

Faktor nr. 1: Datakommunikation, dvs. i hvor høj grad data og/eller kontrol information sendes eller modtages over kommunikationsfaciliteter.

Faktor nr. 2: Distribueret funktionalitet omhandler, hvorvidt applikationen står alene og kører på en enkelt processor eller er distribueret over mange processorer.

Faktor nr. 3: Performance, dvs. om brugeren stiller særlige krav til ydelsen.

Faktor nr. 4: Meget anvendt konfiguration. Scores nul, hvis der ikke er nogen anvendelsesbegrænsninger og fem, hvis den forventede anvendelse kræver en helt speciel indsats for at blive muliggjort, f.eks. hvis der i virkeligheden ikke er plads til den nye applikation inden for den eksisterende konfiguration af systemer.

Faktor nr. 5: Transaktionsmængde. Scores nul, ved få transaktioner og fem, hvis mængden af transaktioner er stor nok til at stresse applikationen.

Faktor nr. 6: On-line inddatering af data. Scores nul, hvis mindre end 15% af transaktionerne er interaktive og fem, hvis mere end 50% er interaktive.

Faktor nr. 7: Slutbruger effektivitet. Scores nul, hvis der ikke er nogen brugere eller de ikke stiller nogen krav og fem, hvis slutbrugernes forventninger kræver en helt speciel indsats for at blive opfyldt.

Faktor nr. 8: On-line opdatering. Scores nul, hvis der ikke er nogen og fem, hvis opdatering er obligatorisk og samtidig svært, f.eks. pga. behov for back-up eller beskyttelse af data mod tilfældig ændring.

Faktor nr. 9: Komplex databehandling. Scores nul, hvis der ikke er nogen databehandling og fem, hvis der kræves omfattende logiske beslutninger, kompliceret matematik eller omfattende sikkerhed.

Faktor nr. 10: Genbrug. Scores nul, hvis der aldrig skal genbruges noget fra den pågældende applikation og fem, hvis der må forudses omfattende genbrug af store dele af systemet.

Faktor nr. 11: Installationslethed. Scores nul, hvis det er uden betydning og fem, hvis der kræves særlig omhu og indsats for at opnå en tilfredsstillende installation.

Faktor nr. 12: Letthed i drift: Scores nul, hvis det er uden betydning og fem, hvis der kræves særlig omhu og indsats for at opnå en tilfredsstillende let drift.

Faktor nr. 13: Multiple steder. Scores nul, hvis der kun planlægges en lokalitet og fem, hvis applikationen skal anvendes mange meget forskellige steder.

Faktor nr. 14: Facilitering af ændringer. Scores nul, hvis brugeren ikke skal kunne ændre noget og fem, hvis brugeren skal kunne lave hyppige ændringer i kontroldata eller tabeller, der vedligeholdes via applikationen.

Summen af scoren for de 14 faktorer indsættes i nedenstående formel:

$$\text{Justeringsmultiplikator} = (\text{SUM}_{\text{af score for 14 faktorer}} * 0,01) + 0,65$$

Herefter anvendes justeringsmultiplikatoren til at komme fra ikke-justerede til justerede Function Points (FP):

$$\text{Justerede FP} = \text{Ikke-justerede FP} * \text{justeringsmultiplikator}$$

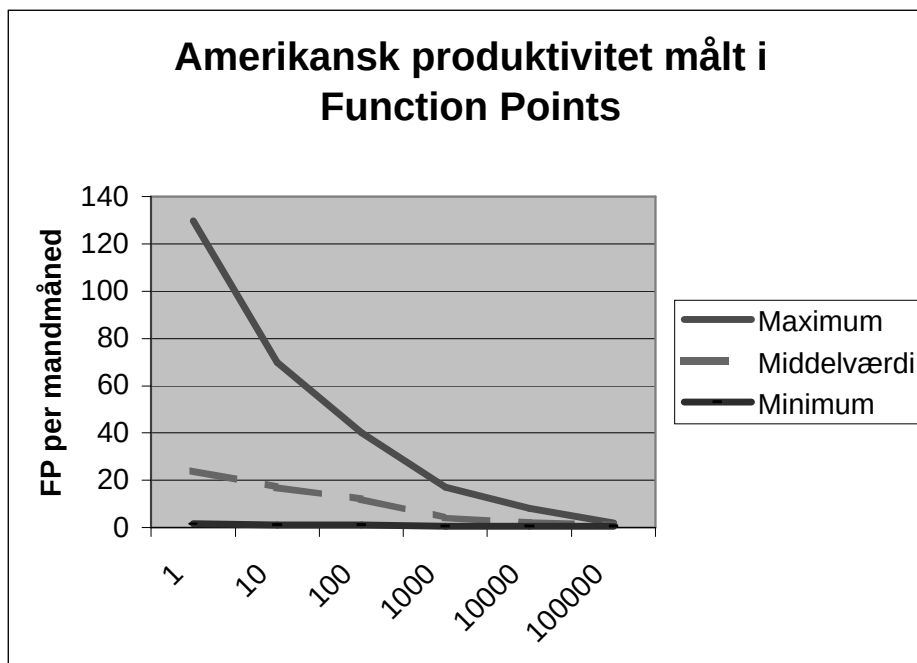
Amerikanske tal for produktivitet målt i Function Points

Capers Jones (1996) og Software Productivity Research (SPR) har omhyggeligt indsamlet data om produktivitet. Nedenstående tabel er et udtryk for produktiviteten målt i Function Points per mandmåned (af 132 effektive arbejdstimer) i forskellige størrelser projekter, fordelt på en række sandsynlighedsintervaller. Tallene i tabellen udtrykker sandsynligheden i % for en given produktivitet i et projekt af en given størrelse. F.eks. er sandsynligheden 35% for en produktivitet mellem 15 og 25 Function Points på et projekt på 100 Function Points.

Størrelse i FP:	1	10	100	1000	10000	100000
Produktivitet						
> 100 FP / mandmåned	10	7	1	0	0	0
75-100 FP / mandmåned	20	15	3	1	0	0
50-75 FP / mandmåned	34	22	7	2	0	0
25-50 FP / mandmåned	22	33	10	3	0	0
15-25 FP / mandmåned	10	17	35	20	2	0
5-15 FP / mandmåned	3	5	26	55	15	20
1-5 FP / mandmåned	1	1	10	15	63	35
< 1 FP / mandmåned	0	0	8	4	20	45

Tabel over sandsynlig produktivitet i procent. Kilde: Capers Jones (1996, side 195)

Tabellen viser tydeligt, at produktiviteten falder stærkt i store projekter. Nedenstående figur, hvor maksimum, middelværdi, og minimum er indtegnet viser tydeligt denne tendens.



Kilde: Capers Jones (1986: 197)

En anden måde at se de amerikanske produktivetsdata er at opdele dem efter industri. Et uddrag af en sådan opdeling er vist i tabellen nedenfor. Udvalget omfatter de industrier, der er mest relevant for medlemmer af Datateknisk Forum, dvs.:

Industri: Produktivitet	MIS	Outsource	Systems
> 100 FP / mandmåned	1	3	0
75-100 FP / mandmåned	4	8	0
50-75 FP / mandmåned	7	12	1
25-50 FP / mandmåned	12	16	10
15-25 FP / mandmåned	18	22	17
5-15 FP / mandmåned	35	27	33
1-5 FP / mandmåned	18	10	27
< 1 FP / mandmåned	5	2	12

Tabel over sandsynlig produktivitet i procent i forskellige industrier. Kilde: Capers Jones (1996, tabel 3.20, side 195)

1. "MIS" lig med Management Information Systems. Gruppen omfatter alle former for administrative edbssystemer, store mainframe såvel som små PC-baserede, udviklet til internt brug.
2. "Outsource" omfatter software produceret på kontrakt, hvad enten der er tale om fast pris, timer & materialer eller andre kunde-leverandør forhold.
3. "Systems" omfatter dels system-software f.eks. operativsystemer, oversættere, og lokalt net, dels indlejret software til apparater, telefon-systemer, luftfart el. lign. Hovedparten af det software, der udvikles i dansk elektronikindustri, falder i denne gruppe.

Af tabellen ses, at netop den type software som mange virksomheder i dansk elektronik- og apparatindustri udvikler, det som Capers Jones kalder "Systems", har en langt større sandsynlighed for en lav produktivitet målt i FP, end de fleste andre industrier (software til militære formål dog undtaget).

Ti tommelfingerregler om Function Points

Capers Jones (1997) har i artiklen "By popular demand: Software estimating rules of thumb" opstillet ti tommelfingerregler, som man kan bruge til nogle meget grove estimater. De ti regler er:

1. Et Function Point svarer til 100 logiske sætninger kildetekst.
2. $FP^{1,15}$ forudsiger antallet af sider dokumentation i relation til projektet.
3. Kravene i projektet vil øges med ca. 1% per måned (såkaldte "Creeping Requirements").
4. $FP^{1,2}$ forudsiger antallet af test cases.
5. $FP^{1,25}$ forudsiger antallet af defekter (fejl) i krav, design, kode og test.
6. Hvert software review, inspektion eller testrunde vil finde 30% af de defekter, der er til stede.
7. $FP^{0,4}$ forudsiger det omtrentlige antal kalendermåneder til udvikling.
8. FP divideret med 150 forudsiger det omtrentlige antal udviklere, der kræves.
9. FP divideret med 500 forudsiger det omtrentlige antal personer, der kræves til vedligeholdelse og opdatering af systemet
10. Antal kalendermåneder (regel 7) ganget med antal udviklere (regel 8) forudsiger antallet af mandmåneder, der kræves til udvikling

Capers Jones slutter sin artikel med en advarsel om tommelfingerregler, som det er relevant at gengive her (1997, side 269):

"Simple rules of thumb continue to be popular, but they are never accurate and are not a substitute for formal estimating methods ... I hope the obvious limitations of these simplistic rules will motivate readers to explore more accurate and powerful methods."